GREEN PALACE HOTEL AI ASSISTANT

Project Report | Local Tool-Calling AI Agent using Ollama

Group member	Student ID
Kahandawala Arachchige Kavindu Lakmal	CCLARK554
J.L.Tharusha	CCLARKE591

Project repository: https://github.com/LakmalPerera55/LLM_Hotel_AI-Agent

Report basis: The group repository on its main branch, the lecturer's Assignment Guide, and observed terminal tests provided by the group.

1. Introduction and problem statement

Hotels receive repeated questions about room categories, prices, availability, facilities, and reservations. Even in a small example hotel, answering these questions manually can be repetitive. The selected scenario is a local AI hotel assistant that helps a guest obtain information and perform simple simulated booking actions using natural-language questions. It is not a commercial reservation service and does not contact a real hotel database.

The Green Palace Hotel AI Assistant runs in a Windows terminal using Python and a locally hosted model through Ollama. It supports nine sample rooms: three Single rooms (101-103), three Double rooms (104-106), and three Family rooms (107-109). Their guest capacities are one, two, and four respectively. The corresponding room prices are Rs. 5,000, Rs. 8,000, and Rs. 12,000 per booking date. Each booking is valid for one full calendar day.

The objectives are to demonstrate an agent that chooses tools based on a user request, checks and updates mock booking records, retrieves hotel information from a local knowledge source, and explains results in simple English. The design deliberately uses a terminal interface and a limited room set so that the tool-calling process remains clear enough to demonstrate and explain in the viva.

2. System architecture

2.1 Components and technologies

The project uses Python for application logic, LangChain to create the agent and expose typed tools, and LangGraph execution behind the agent framework. The response model is **llama3.2** through ChatOllama with temperature set to zero. This is the 3-billion-parameter model family; the group selected it because it supports local tool calling and can run without a paid cloud API. The installed model was approximately 2.0 GB on the test machine. A separate Ollama model, **nomic-embed-text**, produces text embeddings for document retrieval.

Room definitions are held in a Python dictionary and reservations in a Python list. These room-management tools work on mock data. Hotel policies and facilities are read from **docs/info.txt** and stored for retrieval through ChromaDB. The vector database keeps an index on disk, but it is not used to store room reservations.

2.2 Architecture diagram

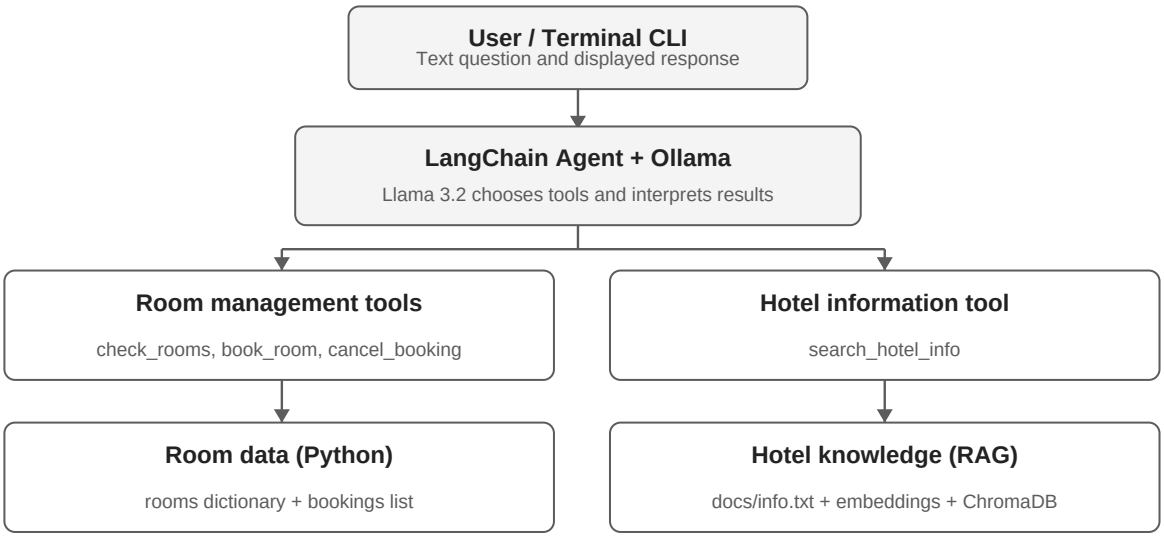

Figure 1. Simplified architecture of the working project. The agent can repeat the model-tool-result cycle before returning an answer.

2.3 Data flow

A guest types a message into the terminal. The conversation history is sent to the agent. The local model decides whether it needs a room tool or the hotel-information search tool. Python runs the selected tool and returns its output to the agent. The agent may request additional calls, then produces a final answer. The terminal also prints the tool name, arguments, and returned content for assessment and debugging. This separation lets the model understand the request while the Python functions enforce the actual booking rules.

3. Tool design and implementation

Four functions are registered with the agent using LangChain's **@tool** decorator. Each tool includes a docstring, typed parameters, and a string response. The descriptions help the LLM choose the relevant function. The tools perform actual checks on local data rather than confirming reservations based only on model-generated text.

Tool and purpose	Inputs	Representative tool result
check_rooms - list prices and date-specific availability	date: str (optional)	Room 104: Double, 2 guest(s), Rs. 8000 per booking date, Available
book_room - create a mock booking	name: str; room_number: int; guests: int; date: str	BOOKING CONFIRMED: Kasun, Room 104, 2 guest(s), 2026-11-10. Price: Rs. 8000.
cancel_booking - remove a matching booking	name: str; room_number: int; date: str	BOOKING CANCELLED: Room 104 for Kasun on 2026-11-10.
search_hotel_info - retrieve hotel knowledge	question: str	Retrieved hotel information from docs/info.txt: ...

3.1 Room availability and booking

The **check_rooms** tool loops through the room dictionary. For a requested date, it compares each room number and date against the bookings list and labels the room Available or Booked. When no date is provided, it lists room categories and prices but does not claim that the rooms are available on any specific date. The **book_room** tool verifies that the guest name is present, the room exists, the date is valid, and the guest count does not exceed capacity. It rejects a second booking for the same room and date. An accepted booking adds a dictionary containing the name, room number, guest count, and date to the list.

3.2 Cancellation and information search

The **cancel_booking** tool checks the requested guest name, room, and date before removing an existing matching entry. It uses case-insensitive name comparison and returns a clear message if no matching reservation exists. The **search_hotel_info** tool checks that the question is not blank, performs semantic retrieval through ChromaDB, and returns the most relevant passages from the hotel information file. It is intended for facilities, room features, food, services, and policies, not for live prices or booking status.

3.3 Input validation

Dates are checked with datetime parsing and format comparison using YYYY-MM-DD; impossible calendar dates are rejected. Missing names, unknown room numbers, unsuitable guest counts, and absent matching cancellations return explanatory messages instead of silently succeeding. The implementation is a teaching demonstration using simulated bookings; it does not connect to a live hotel reservation platform.

4. Embedding-based RAG implementation

Retrieval-Augmented Generation (RAG) lets the assistant answer hotel information questions using a supplied document instead of relying only on the model's general knowledge. The knowledge source is **docs/info.txt**, which describes the hotel, its room features, pool, Wi-Fi, parking, breakfast hours, policies, bookings, and guest services. For example, the document states that the outdoor swimming pool is open from 7:00 AM to 7:00 PM and breakfast is served from 7:00 AM to 10:00 AM.

During startup, **build_vector_store()** reads the text file, checks that it exists and is not empty, and divides its text into chunks using RecursiveCharacterTextSplitter with a chunk size of 500 characters and overlap of 80. OllamaEmbeddings with **nomic-embed-text** converts those chunks to numerical representations. A Chroma collection called **green_palace_hotel** stores the embeddings in the **chroma_db** directory. The search tool uses similarity_search with **k=3** to obtain three relevant chunks for each question and passes the retrieved text back to the agent.

The current startup function deletes existing Chroma documents before adding the new chunks again. This ensures that the current information file is indexed, although it repeats embedding work whenever the program starts. The model can also misunderstand a retrieved passage: similarity-based retrieval finds related text, but it does not guarantee that a passage directly answers every question. These are recognised design limitations rather than claims of perfect accuracy.

5. Agent loop and prompt design

5.1 Tool-call workflow

The program constructs a LangChain agent with ChatOllama and the four tools. In each loop iteration, the terminal takes the user's question, appends it to a conversation history, and calls agent.invoke. The agent may respond with a tool request; the framework executes the chosen function and sends the result back to the model. This can repeat for a multi-step request. The program prints the final message after the agent has finished, then retains the returned messages as its new history. The session ends when the user types exit.

A **recursion_limit of 15** is supplied to the agent invocation as a guard against unbounded graph execution. This number refers to graph steps, not a promise of exactly 15 tool calls. The system also uses a system prompt that identifies the hotel-assistant role, assigns different questions to the four tools, specifies one-calendar-date bookings, requests missing details, and tells the model not to announce unconfirmed reservations or output tool-call commands as plain text.

5.2 Memory and observability

The program keeps a **history** list during the terminal session so follow-up questions can refer to earlier dates or rooms. The history is not persistent across program restarts. Tool-call logging uses AIMessage and ToolMessage records returned by the agent: it prints the called tool and its arguments, then the tool results. Logging makes it possible to verify which tools the agent requested and what they returned. These records are useful when reviewing multi-step requests and explaining the workflow.

6. Testing and results

The group performed manual terminal tests with the local Ollama agent. The outcomes below distinguish behaviour seen in shared terminal logs from behaviour that is implemented in the code but not established by a recorded live example. This distinction matters because a function existing in source code does not prove that the model always uses it correctly.

Test	Evidence and observed outcome	Assessment
Availability check	Room 104 reported Available for 2026-11-10.	Observed: passed
Room booking	book_room returned BOOKING CONFIRMED for Kasun, two guests, room 104.	Observed: passed
Recheck after booking	A later check_rooms result reported room 104 as Booked for the same date.	Observed: passed
Three-step request	The terminal trace showed check_rooms, book_room, and check_rooms in one request.	Tool-call trace observed
Hotel knowledge search	A swimming-pool question retrieved relevant policy information and a correct pool answer.	Observed: passed
RAG approach revision	The earlier keyword approach was replaced by embedding-based document retrieval.	Implementation revised
Duplicate / invalid / cancel paths	Rules and explanatory responses are present in Python functions.	Code reviewed; further live demonstration needed

6.1 What worked

The separately submitted booking questions produced consistent results in the terminal: room 104 was available, the booking for Kasun was confirmed, and a subsequent request reported it as booked. The trace also demonstrated that the same agent can call several tools from one request. The hotel-information tool retrieved the swimming pool passage from the Chroma-backed knowledge base. These tests establish that the basic tools, local agent, and displayed tool traces are functional in the observed runs.

6.2 Development challenges and improvements

An early version of the hotel-information search used keyword-based matching. This approach depends strongly on shared words between the question and the source text, so questions using different wording can be harder to match. The team replaced it with embedding-based RAG using OllamaEmbeddings and ChromaDB. This was a retrieval-design improvement: no numerical accuracy comparison between the two approaches was recorded. After the change, the observed swimming-pool test retrieved the relevant information and the assistant correctly reported the facility and opening hours.

The system prompt was also refined to specify when the assistant should use each tool and to require confirmation messages to be based on tool results. Tool output logging was kept visible so the group could inspect the requested arguments and returned values during development. The tests listed in Table 2 document the observed outcomes; a tool-call trace demonstrates that an operation was requested, while a final user-facing reply should be assessed separately.

7. Limitations and future improvements

The most important limitation is booking persistence. The booking records are stored in a Python list, so they are lost when the process closes. A practical next improvement is to save reservations in a file or an SQLite database and reload them when the system starts. This would preserve room availability after restarts and make repeated testing less dependent on a single active session.

In the future, a simple web interface could be added to make the hotel assistant easier to use. Users could check room availability, make mock bookings, cancel reservations, and ask about hotel facilities through buttons and input fields instead of typing commands in the terminal.

8. Individual contributions

Both students discussed the project scenario and planned the hotel-assistant features together. Implementation responsibilities were divided by tools and the members collaborated on integration, testing, and improvements. The following table describes the team's reported division of work.

Student	Primary work	Shared work
J.L.Tharusha (CCLARKE591)	Room availability and booking tools; agent/terminal integration; tool-call logging.	Scenario planning, design discussions, tests and revisions.
Kahandawala Arachchige Kavindu Lakmal (CCLARK554)	Cancellation and hotel-information tools; embedding-based RAG and ChromaDB.	Scenario planning, design discussions, tests and revisions.

9. Conclusion

The Green Palace Hotel AI Assistant demonstrates a local tool-calling agent that combines structured mock booking operations with document retrieval. The application uses four typed Python tools, Ollama models, LangChain agent execution, conversation history, and visible tool logging. Manual tests show successful room checks, booking confirmation, a changed availability status, and hotel-information retrieval. The move from keyword-based matching to embedding-based RAG also shows how the information-search design evolved during development. The project provides a practical educational prototype that could later be expanded with database-backed booking storage.

Appendix A. AI assistance declaration

ChatGPT was consulted during development to identify errors, debug installation and agent behaviour, review code, detect weaknesses, suggest improvements, and explain possible fixes. It also assisted with proposed code revisions and the organisation of report material. Group members reviewed and tested the implementation and remain responsible for understanding and presenting the submitted work.